

AI-DRIVEN ADAPTIVE CODING ASSISTANCE AND INTERVIEW PREPARATION PLATFORM

Project Id: 25-26J-364

Project Proposal Report

Himansha L.M.H – IT22601360

B.Sc. (Hons) in Information Technology Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

August 2025

AI-DRIVEN ADAPTIVE CODING ASSISTANCE AND INTERVIEW PREPARATION PLATFORM

Project Id: 25-26J-364

Project Proposal Report

B.Sc. (Hons) in Information Technology Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

August 2025

DECLARATION PAGE OF THE CANDIDATES & SUPERVISOR

We declare that this is our own work and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Group Member Name	Student ID	Signature
Himansha L.M.H	IT22601360	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

Signature of the supervisor:



Date: 31/08/2025

ABSTRACT

The transition from academic programming education to industry-level software development presents significant challenges for university students, primarily due to the disconnect between theoretical knowledge acquisition and practical concept application. Current educational platforms fail to provide comprehensive feedback on the underlying theoretical concepts embedded within student code, limiting their ability to understand the "why" behind their programming solutions. This research proposes the development of a Code Concept Extractor—an intelligent static analysis system that combines Abstract Syntax Tree (AST) parsing with Natural Language Processing (NLP) models to automatically identify, extract, and visualize fundamental computer science concepts from source code.

The proposed system addresses critical gaps in programming education by providing automated concept-level feedback, visual representation of theoretical principles, and structured knowledge extraction from student implementations. Unlike existing code analysis tools that focus primarily on syntax validation or performance optimization, this research introduces a novel approach that bridges theoretical computer science education with practical programming skills through intelligent concept recognition and educational visualization.

The Code Concept Extractor employs a multi-layered architecture incorporating static code analysis engines, transformer-based NLP models, and dynamic visualization generators to parse user-submitted code across multiple programming languages. The system identifies object-oriented programming patterns, data structure implementations, algorithmic paradigms, and design pattern applications, presenting findings through interactive diagrams, conceptual summaries, and structured knowledge maps. This research contributes to the field of educational technology by providing an automated, scalable solution for concept-driven code analysis that enhances student understanding of theoretical computer science principles through practical code examination.

TABLE OF CONTENTS

Declaration page of the candidates & supervisor.....	3
Abstract	4
TABLE OF CONTENTS	5
LIST OF FIGURES.....	6
LIST OF ABBREVIATIONS	6
1. Introduction	7
1.1 Background & Literature survey.....	7
1.2 Research Gap.....	8
1.3 Research Problem.....	9
2. Objectives.....	12
2.1 Main Objective.....	12
2.2 Sub-Objectives	12
3. Methodology	14
3.1 Implementation Technologies and Tools	18
3.2 Evaluation Methodology	19
3.3 Data Collection Procedures.....	20
3.4 Validation and Verification Procedures	21
3.5 Ethical Considerations and Privacy Protection	22
3.6 Integration with Overall Platform Architecture	22
4. Description of Personal and Facilities.....	23
5. Work Breakdown Structure.....	26
6. Gantt Chart	27
7. References	28

LIST OF FIGURES

FIGURE 1 OVERAL SYSTEM ARCHITECTURE DIAGRAM 14

FIGURE 2 CODE CONCEPT EXTRACTOR SYSTEM ARCHITECTURE DIAGRAM..... 15

FIGURE 3 SYSTEM INTERACTION FLOW 18

FIGURE 4 5. WORK BREAKDOWN STRUCTURE..... 26

FIGURE 5 GANTT CHART..... 27

LIST OF ABBREVIATIONS

Abbreviation	Description
AST	Abstract Syntax Tree
AI	Artificial Intelligence
DSR	Design Science Research
OOP	Object Oriented Programming
NLP	Natural Language Processing
SUS	System Usability Scale

1. INTRODUCTION

1.1 Background & Literature survey

The landscape of computer science education has evolved significantly over the past decade, with increasing emphasis on bridging the gap between theoretical knowledge and practical application. Traditional programming education methodologies often focus on syntax mastery and algorithm implementation without adequately addressing the underlying theoretical concepts that govern effective software design and development [1].

Static code analysis has emerged as a fundamental technique in software engineering, primarily utilized for bug detection, security vulnerability identification, and code quality assessment. Traditional static analysis tools such as SonarQube, ESLint, and PMD focus on syntax validation, code smell detection, and performance optimization rather than educational concept extraction [2].

Recent advances in Natural Language Processing, particularly transformer-based models and semantic understanding techniques, have opened new possibilities for intelligent code analysis beyond traditional syntactic approaches. Research has demonstrated the effectiveness of large language models in understanding code semantics and generating human-readable explanations of complex algorithms [3]. Similarly, studies have explored the application of NLP techniques to source code analysis, revealing significant potential for automated code comprehension and explanation systems [4].

Educational technology research has consistently highlighted the importance of immediate, concept-level feedback in programming education. Studies have demonstrated that students benefit significantly from automated feedback systems that go beyond simple correctness checking to provide insights into programming concepts and design principles [5][6]. However, existing educational platforms such as LeetCode, HackerRank, and CodeSignal primarily focus on problem-solving correctness rather than conceptual understanding validation.

The integration of visualization techniques in programming education has shown promising results in enhancing student comprehension of abstract concepts. Visual representations of data structures, algorithms, and program execution have been demonstrated to significantly improve student understanding and retention of complex programming concepts [7]. However, most existing visualization tools require manual creation or are limited to specific programming constructs, lacking the flexibility and automation necessary for comprehensive educational support.

Recent developments in Abstract Syntax Tree (AST) analysis and code parsing technologies have provided robust foundations for automated code understanding systems. Tools such as Tree-sitter, ANTLR, and language-specific parsers enable precise structural analysis of source code across multiple programming languages [8]. These technologies, combined with machine learning approaches, offer unprecedented opportunities for developing intelligent educational systems capable of understanding and explaining code at a conceptual level.

1.2 Research Gap

Although significant progress has been made in automated code analysis and educational technologies, several **critical limitations** remain when it comes to helping students **understand the theoretical concepts** embedded in their programming work. These gaps illustrate the need for an advanced system like the **Code Concept Extractor**:

Lack of Concept Extraction

Current static analysis and automated grading tools primarily emphasize error detection, syntax validation, and code efficiency analysis. While these are useful for debugging and improving technical correctness, they do not reveal whether the student has implemented theoretical constructs such as recursion, inheritance, modularity, or stack-based memory management. For example, a tool might detect that a loop is inefficient but will not explain that the student’s recursive solution reflects the use of the divide-and-conquer paradigm, which is a fundamental computer science principle. This omission leaves students without a clear understanding of the theories they are applying in practice.

Absence of Theory-Oriented Feedback

Most existing educational platforms—such as LeetCode, HackerRank, and Codeforces—focus almost exclusively on whether the code passes test cases. Their feedback is oriented toward correctness and performance, not conceptual mastery. Consequently, a student who successfully implements a solution may receive validation for correctness but no guidance on the underlying algorithmic paradigm, design principle, or theoretical concept demonstrated in their work. This narrow form of feedback risks reinforcing shallow problem-solving approaches without cultivating the deeper conceptual understanding necessary for long-term learning.

Inadequate Visualization and Reporting

Visualization has been widely recognized as a powerful pedagogical tool, as it helps learners connect abstract concepts to tangible representations. Existing visualization tools such as Python Tutor provide some functionality but are limited to execution tracing and predefined rules, making them less flexible for diverse programming approaches. Furthermore, there is no system capable of automatically generating comprehensive, downloadable reports that summarize the concepts used in a student's code. Without such reports, students lack the ability to reflect on their conceptual progress over time, and instructors are left without scalable tools to evaluate concept mastery across cohorts.

Cross-Language Theory Mapping

Programming principles such as object-oriented inheritance, modular design, or recursion are universal across languages, but they manifest differently depending on syntax and implementation details. At present, no system exists that can map and highlight these language-agnostic concepts for learners. This limitation results in students perceiving concepts as being language-specific, which hinders their ability to transfer knowledge across paradigms. For example, recursion in C may appear structurally different from recursion in Python, but the theoretical principle remains the same. A lack of tools that can generalize these concepts contributes to fragmented learning experiences.

Integration with Learning Workflows

Finally, existing tools often function in isolation, separate from the broader educational ecosystem. Many are used as practice platforms but lack integration with assignments, grading systems, and personalized learning pathways. As a result, instructors cannot easily embed concept analysis into coursework, and students cannot receive continuous, personalized feedback on their conceptual progress. A truly effective educational tool must integrate seamlessly into assignments, allowing students to submit their code, automatically receive concept-level reports, and track their learning progress over time.

1.3 Research Problem

The primary research problem addressed by this study is the **lack of intelligent, automated systems capable of extracting, identifying, and presenting theoretical computer science concepts embedded in student-written code.**

In traditional programming education, students often learn how to solve problems procedurally but fail to recognize the deeper theoretical ideas they are applying. This

phenomenon, referred to in this study as “conceptual blindness,” occurs when students successfully implement a solution (e.g., through recursion, dynamic programming, or inheritance) without realizing the theoretical foundation underpinning their implementation. While such students may become adept at solving familiar problems, they frequently struggle to transfer knowledge to novel contexts, as they lack the conceptual awareness required for adaptive problem-solving.

Existing tools exacerbate this issue by focusing narrowly on syntax correctness, style compliance, and test case validation. Although these features are valuable, they fail to provide explanations of theoretical significance. For instance, a system might indicate that a recursive function executes correctly but will not explain that the function demonstrates the divide-and-conquer paradigm, a principle with broad applications in algorithm design. This results in a learning environment where students receive superficial validation but not deep educational feedback.

Visualization, an important element of programming pedagogy, has proven effective in helping students grasp abstract computational processes. However, most existing visualization tools are static, manually configured, or limited to specific constructs, meaning they cannot adapt to arbitrary student code. Furthermore, no current system combines real-time visualization with downloadable concept reports, leaving students without portable learning artifacts and instructors without scalable assessment tools. This limitation reduces opportunities for reflection, assessment, and structured knowledge-building.

Another key problem is the language-specificity of existing tools. Since different programming languages represent concepts differently, students often fail to recognize the shared, underlying principles. Without cross-language concept recognition, students may become overly reliant on a single syntax, thereby weakening their ability to generalize knowledge. This not only restricts their learning but also limits their adaptability as future software engineers who must often switch between languages and paradigms.

To address these challenges, this research proposes the development of a Code Concept Extractor system designed to provide real-time, concept-level feedback to students. By allowing learners to paste, type, or upload code, the system will automatically analyze the implementation, identify theoretical concepts (such as recursion, modularity, or object-oriented inheritance), and generate both visualizations and downloadable reports. These reports can serve as a resource for students to reflect on their learning, for instructors to assess progress, and for institutions to integrate into curriculum-wide educational strategies.

Ultimately, this research seeks to bridge the gap between practical programming practice and conceptual understanding, ensuring that students not only know how to code but also understand why their code works in terms of fundamental computer science principles. By doing so, the proposed system will contribute to a more effective, scalable, and concept-focused model of programming education.

2. OBJECTIVES

2.1 Main Objective

The primary objective of this study is to design and implement a **Code Concept Extractor** system that automatically analyzes source code and identifies the underlying theoretical programming concepts embedded within it. This system aims to bridge the gap between practical coding exercises and theoretical knowledge by mapping code snippets to concepts such as Object-Oriented Programming (OOP) principles, data structures, algorithms, and design patterns. By providing students with structured outputs in the form of summaries, diagrams, and tables, the proposed solution seeks to enhance the process of understanding coding theories and foster a more effective learning experience. Ultimately, this system will serve as an educational tool that allows university students to not only practice coding but also strengthen their grasp of theoretical foundations in computer science.

2.2 Sub-Objectives

To achieve the above objective, the study will focus on the following sub-objectives:

1. Source Code Input and Processing

One sub-objective is to enable students to provide source code through multiple modes such as cloning from a repository, pasting into the system, or typing directly. The system will preprocess the code to clean formatting errors and ensure compatibility across different programming languages. This step ensures that students can seamlessly integrate their work into the tool without facing technical barriers.

2. Automated Concept Identification

Another sub-objective is to implement static code analysis combined with Natural Language Processing (NLP) techniques to identify programming concepts embedded in the source code. For instance, the system should detect the presence of OOP principles like inheritance, polymorphism, and encapsulation, as well as the use of data structures such as arrays, linked lists, or trees. This allows the system to act as a bridge between abstract theoretical knowledge and its real-world coding implementations.

3. Structured Representation of Extracted Concepts

A further sub-objective is to present the identified concepts in an organized and easy-to-understand format. The system will generate summaries, visual diagrams (e.g., UML class diagrams), and tabular representations of concepts. This structured output will not only enhance the student's comprehension but also make it easier to identify gaps in their understanding and study more effectively.

4. Report Generation and Download

Another sub-objective is to allow students to export the analyzed results as a comprehensive report. The report will summarize the detected concepts, provide explanations, and include visual representations. Downloading the report in standard formats such as PDF or DOCX will help students retain material for offline learning, assignment submission, or interview preparation.

5. Improved Learning and Evaluation Support

Finally, the system aims to function as both a self-learning and evaluation tool. By mapping student-written code to theoretical foundations, it can highlight strengths and weaknesses in their coding practices. This supports independent learning and also provides educators with a potential tool to evaluate students' conceptual grasp beyond simply testing coding syntax.

Through these objectives, the Code Concept Extractor aims to provide an innovative and practical learning support system that directly addresses the challenges faced by students in connecting theoretical computer science knowledge with hands-on coding practices.

3. METHODOLOGY

Research Design and Approach

This research employs a Design Science Research (DSR) methodology combined with experimental validation and user-centered design principles. The methodology follows an iterative development cycle incorporating system design, implementation, testing, and refinement phases to ensure the Code Concept Extractor meets both technical and educational requirements.

System Architecture Overview

The Code Concept Extractor operates as an integral component within the larger AI-Driven Adaptive Coding Assistance and Interview Preparation Platform. The overall system architecture follows a microservices design pattern, enabling modular development and deployment while facilitating seamless integration between the four core components: Code Concept Extractor, Code Optimization and Refactoring, Weakness Detection and Guidance, and AI Interview Simulator.

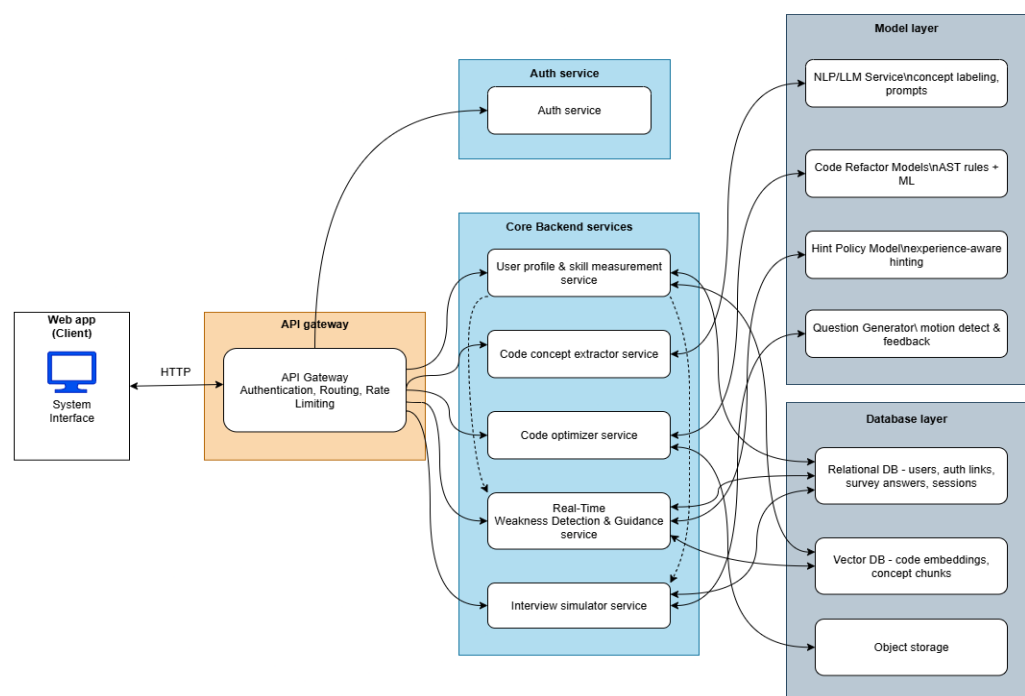


Figure 1 Overall System Architecture Diagram

The platform architecture implements a service-oriented approach where each research component operates as an independent service while sharing common infrastructure elements including user management, authentication services, and data persistence layers. This design ensures scalability, maintainability, and enables parallel development across research team members.

Code Concept Extractor System Architecture

The Code Concept Extractor system architecture consists of five primary modules: Input Processing Module, Static Analysis Engine, Natural Language Processing Module, Concept Recognition Module, and Visualization and Reporting Module. Each module is designed to handle specific aspects of the concept extraction pipeline while maintaining loose coupling to enable independent development and testing.

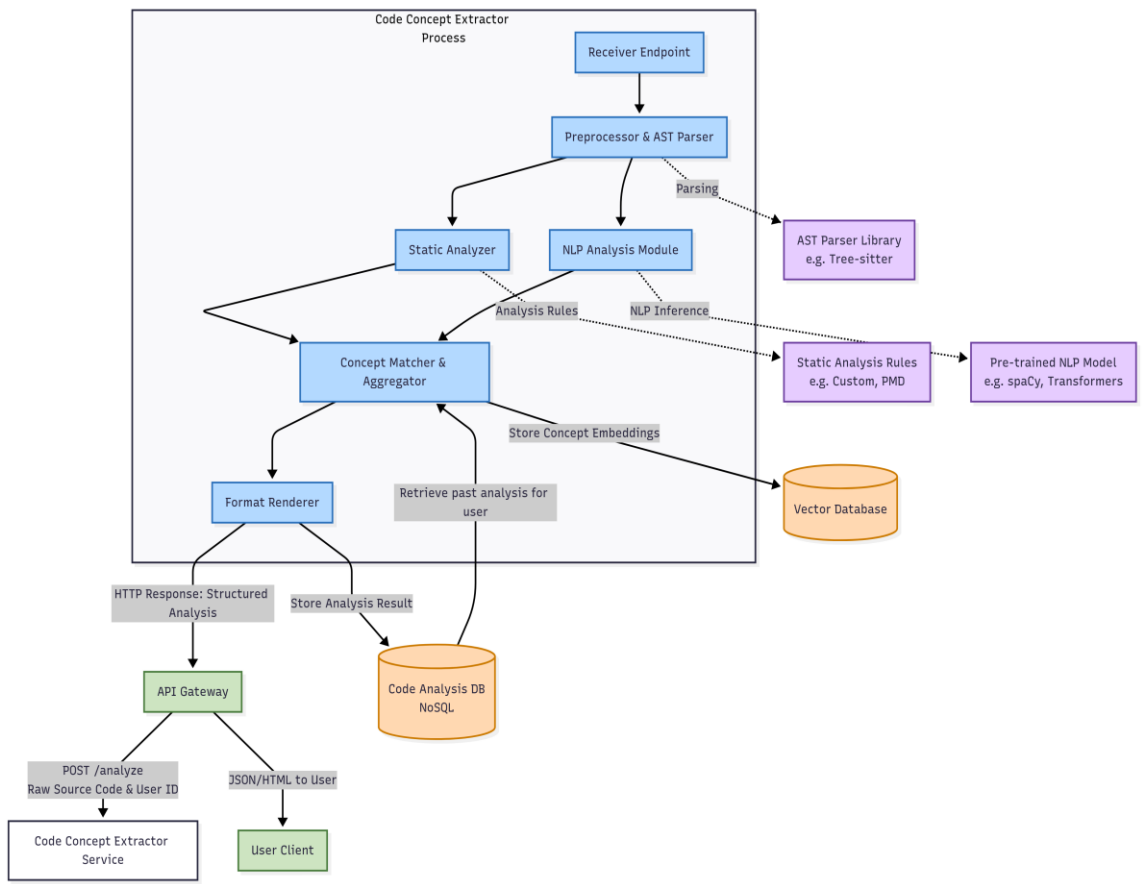


Figure 2 Code Concept Extractor System Architecture Diagram

Input Processing Module

The Input Processing Module serves as the entry point for user code submissions and implements comprehensive input validation and preprocessing functionality. This module supports multiple input methods including direct code typing, clipboard pasting, and file uploads supporting common programming file formats (.py, .java, .cpp, .js, .c, .h).

The module implements syntax validation using language-specific parsers to ensure code correctness before proceeding with analysis. Input preprocessing includes comment extraction for documentation analysis, whitespace normalization, and encoding standardization to UTF-8 format. Error handling mechanisms provide detailed feedback for syntax errors, unsupported language constructs, and file format issues.

Static Analysis Engine

The Static Analysis Engine forms the core of the concept extraction system, implementing multi-language parsing capabilities and deep code structure analysis. The engine utilizes Tree-sitter parsing library [9] for generating language-agnostic Abstract Syntax Trees (ASTs) supporting Python, Java, C++, JavaScript, and C programming languages.

The engine implements visitor pattern algorithms for systematic AST traversal and code pattern recognition. Symbol table generation algorithms track variable declarations, scope relationships, and identifier usage patterns throughout the codebase. Control flow graph generation enables analysis of program execution paths, loop structures, and conditional logic patterns.

Advanced analysis capabilities include dependency analysis for identifying relationships between classes and functions, complexity metrics calculation using cyclomatic complexity and Halstead metrics, and design pattern recognition for identifying common programming paradigms such as Singleton, Factory, Observer, and Strategy patterns.

Natural Language Processing Module

The Natural Language Processing Module integrates state-of-the-art transformer-based models specifically trained for code understanding tasks. The module utilizes CodeBERT [10] and GraphCodeBERT [11] pre-trained models for generating contextual embeddings of code structures and semantic relationships.

Custom fine-tuning procedures adapt the pre-trained models to the specific task of concept identification using curated datasets containing code samples annotated with corresponding theoretical concepts. The module implements text preprocessing pipelines converting code structures into tokenized representations suitable for transformer model processing.

Semantic similarity algorithms compare extracted code features with predefined concept templates to identify theoretical concept implementations. The module generates confidence scores for each identified concept using ensemble methods combining multiple model predictions and rule-based validation.

Concept Recognition Module

The Concept Recognition Module implements hybrid approaches combining rule-based pattern matching with machine learning classification for robust concept identification. Rule-based systems identify explicit implementations of fundamental concepts such as data structures (arrays, linked lists, trees, graphs), algorithms (sorting, searching, dynamic programming), and programming paradigms (object-oriented principles, functional programming constructs).

Machine learning classifiers utilize Random Forest and Support Vector Machine algorithms trained on labeled code datasets to recognize implicit concept applications and complex design patterns. The module implements hierarchical classification supporting multi-level concept categorization from basic programming constructs to advanced algorithmic concepts.

Confidence scoring mechanisms evaluate identification reliability using multiple validation techniques including cross-validation with expert annotations, consistency checking across similar code patterns, and threshold-based filtering to minimize false positive identifications.

Visualization and Reporting Module

The Visualization and Reporting Module generates comprehensive visual representations and detailed reports of identified concepts. Dynamic diagram generation utilizes D3.js library [12] for creating interactive visualizations including class relationship diagrams, data structure representations, and algorithm flow charts.

The module implements template-based report generation creating structured documents containing concept summaries, code snippet explanations, theoretical background information, and learning recommendations. Export functionality supports multiple formats including PDF reports, interactive HTML presentations, and JSON data structures for external system integration.

System Interaction Flow

The Code Concept Extractor follows a systematic processing workflow designed to maximize accuracy while maintaining efficient processing times. The interaction flow is documented through a comprehensive sequence diagram illustrating the communication patterns between system components and user interactions.

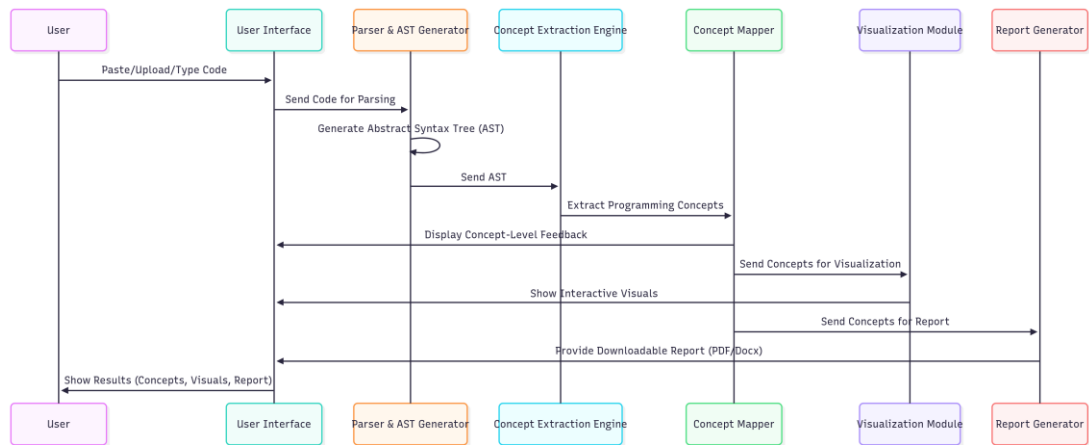


Figure 3 System Interaction Flow

The processing workflow begins with user code submission through the web interface, followed by input validation and preprocessing. The Static Analysis Engine performs initial code parsing and structure analysis, generating ASTs and extracting syntactic features. Processed code representations are then forwarded to the NLP Module for semantic analysis and embedding generation.

The Concept Recognition Module receives both syntactic and semantic features, applying hybrid classification algorithms to identify theoretical concepts. Results undergo validation and confidence scoring before being forwarded to the Visualization Module for report generation. The complete analysis results, including visualizations and downloadable reports, are presented to the user through the web interface.

3.1 Implementation Technologies and Tools

Programming Languages and Frameworks

The system backend is implemented using Python 3.9+ leveraging its extensive ecosystem of scientific computing and machine learning libraries. The web interface utilizes React.js framework with TypeScript for type-safe frontend development. RESTful API

implementation uses FastAPI framework providing automatic API documentation and high-performance request handling.

Static Analysis Libraries

Multi-language parsing capabilities are implemented using Tree-sitter library providing robust parsing for Python, Java, C++, JavaScript, and C. Additional language-specific analysis tools include Pylint and ast module for Python analysis, JavaParser for Java code processing, and Clang Static Analyzer for C/C++ analysis.

Machine Learning and NLP Libraries

Natural language processing capabilities utilize Transformers library [13] for implementing CodeBERT and GraphCodeBERT models. Scikit-learn library provides machine learning algorithms for classification tasks, while NLTK and spaCy libraries support text preprocessing and feature extraction tasks.

Visualization and Reporting Tools

Dynamic visualization generation utilizes D3.js library for interactive diagram creation, Graphviz for automated graph layout generation, and Matplotlib for statistical visualization. Report generation implements ReportLab library for PDF creation and Jinja2 templating engine for HTML report generation.

Database and Deployment Infrastructure

System data persistence utilizes PostgreSQL database with SQLAlchemy ORM for robust data management. Cloud deployment implements Docker containerization with Kubernetes orchestration for scalable system deployment. Redis caching system optimizes performance for frequently accessed code analysis results.

3.2 Evaluation Methodology

Accuracy Evaluation Framework

System accuracy evaluation implements multi-faceted assessment approaches measuring concept identification precision, recall, and F1-scores across different programming languages and concept categories. Ground truth datasets are established through expert annotation by computer science faculty members with expertise in relevant theoretical areas.

Evaluation datasets include carefully curated code samples representing various difficulty levels, programming paradigms, and concept implementations. Statistical analysis includes cross-validation procedures, inter-annotator agreement measurements using Cohen's kappa coefficient, and comparative analysis with existing code analysis tools.

Educational Effectiveness Assessment

Educational impact evaluation utilizes controlled experimental design comparing learning outcomes between students using the Code Concept Extractor and traditional learning methods. Pre- and post-assessment measurements evaluate student understanding of theoretical concepts using standardized computer science concept inventories.

Longitudinal studies track student progress over academic semesters, measuring concept mastery development, code comprehension improvement, and theoretical knowledge retention. Qualitative assessment includes structured interviews and focus groups exploring user experience, learning preferences, and system utility perceptions.

Usability and Performance Evaluation

Usability evaluation follows established Human-Computer Interaction (HCI) evaluation frameworks including System Usability Scale (SUS) measurements and task completion time analysis. User experience assessment examines interface intuitiveness, feature discoverability, and overall satisfaction metrics.

Performance evaluation measures system response times, processing accuracy under various load conditions, and scalability characteristics supporting concurrent user access. Stress testing validates system behavior under peak usage scenarios and identifies performance bottlenecks requiring optimization.

3.3 Data Collection Procedures

Quantitative Data Collection

Quantitative data collection encompasses system performance metrics, accuracy measurements, and user interaction analytics. Automated logging systems capture processing times, concept identification frequencies, error rates, and user engagement patterns. Learning outcome measurements utilize validated assessment instruments measuring concept understanding improvements.

Statistical data analysis employs descriptive statistics, inferential testing using appropriate parametric and non-parametric methods, and effect size calculations for educational impact assessment. Data collection procedures ensure statistical power requirements are met for meaningful hypothesis testing.

Qualitative Data Collection

Qualitative data collection utilizes semi-structured interviews with students, educators, and domain experts exploring system usability, educational effectiveness, and integration challenges. Focus group discussions examine user experience perspectives, learning preference variations, and system improvement recommendations.

Expert review sessions involve computer science faculty evaluating concept identification accuracy, educational relevance, and theoretical completeness. Case study development documents specific usage scenarios, learning outcomes, and system impact on educational practices.

3.4 Validation and Verification Procedures

Technical Validation

Technical validation ensures system accuracy and reliability through comprehensive testing procedures. Concept identification accuracy validation compares system outputs with expert manual analysis using established inter-rater reliability measures. Cross-language validation confirms concept recognition consistency across different programming paradigms.

Algorithm validation tests ensure correct implementation of static analysis algorithms, NLP processing pipelines, and machine learning classification models. Performance validation confirms system meets specified response time requirements and scalability targets.

Educational Validation

Educational validation assesses system effectiveness in supporting student learning through controlled experimental studies. Randomized controlled trials compare learning outcomes between experimental groups using the Code Concept Extractor and control groups using traditional learning methods.

Concept mastery assessment utilizes validated instruments measuring student understanding of data structures, algorithms, object-oriented principles, and software design concepts. Longitudinal tracking evaluates learning retention and concept application in subsequent coursework.

3.5 Ethical Considerations and Privacy Protection

Research procedures comply with institutional review board requirements for human subjects research involving student participants. Comprehensive informed consent procedures ensure participants understand data collection practices, research purposes, and voluntary participation rights.

Data privacy protection implements encryption for data transmission and storage, anonymization protocols for research analysis, and secure access controls for user account management. User code submissions are processed securely with options for automatic deletion after analysis completion.

Student privacy protection ensures individual performance data remains confidential while enabling aggregate analysis for research purposes. Participation withdrawal procedures allow users to remove data from research studies while maintaining access to educational system features.

3.6 Integration with Overall Platform Architecture

The Code Concept Extractor integrates with the broader platform through standardized API interfaces enabling data sharing with other research components. Integration points include sharing extracted concept information with the Weakness Detection and Guidance component for targeted learning recommendations and providing concept mastery data to the AI Interview Simulator for personalized question generation.

Shared database schemas ensure consistent data representation across platform components while maintaining modular development independence. Event-driven architecture enables real-time communication between components for coordinated user experience delivery.

The integration design supports future platform expansion and additional research component integration while maintaining system performance and user experience quality standards.

4. DESCRIPTION OF PERSONAL AND FACILITIES

Personal Qualifications and Expertise

Academic Background The researcher possesses a strong foundation in Information Technology with specialized focus on software engineering, artificial intelligence, and educational technology systems. Current academic standing as an undergraduate student in Information Technology provides direct insight into the challenges faced by students in transitioning from theoretical knowledge to practical programming skills. This firsthand experience with programming education gaps provides valuable perspective for developing user-centered educational solutions.

Technical Competencies

- **Programming Languages:** Proficient in Python, Java, C++, and JavaScript—the four target languages for the Code Concept Extractor system
- **Web Development:** Comprehensive experience with full-stack web development using modern frameworks (React.js, Node.js, Express.js) essential for building the web-based platform
- **Machine Learning and NLP:** Practical experience with machine learning frameworks (TensorFlow, PyTorch, scikit-learn) and natural language processing libraries (NLTK, spaCy, Transformers) required for concept recognition algorithms
- **Static Code Analysis:** Familiarity with AST parsing, code analysis tools, and software engineering principles necessary for implementing the core analysis engine
- **Database Management:** Experience with both SQL (PostgreSQL, MySQL) and NoSQL (MongoDB) database systems for storing user data, code submissions, and extracted concepts
- **Cloud Computing:** Knowledge of cloud platforms (AWS, Google Cloud, Azure) for scalable system deployment and management

Research and Development Skills

- **Software Engineering Methodology:** Understanding of agile development practices, version control systems (Git), and collaborative development workflows essential for systematic system development

- **User Experience Design:** Experience with user interface design principles and usability testing methodologies crucial for creating intuitive educational interfaces
- **Data Analysis:** Proficiency in statistical analysis tools (R, Python pandas, numpy) for evaluating system accuracy and educational effectiveness
- **Technical Documentation:** Strong technical writing skills for creating comprehensive system documentation, research publications, and user guides

Institutional Facilities and Resources

Computing Infrastructure

- **Development Workstations:** Access to high-performance computing workstations with multi-core processors, adequate RAM (16GB+), and SSD storage for efficient development and testing
- **University Computer Labs:** 24/7 access to computer science department laboratories equipped with latest development tools, multiple programming environments, and high-speed internet connectivity
- **Cloud Computing Resources:** University-provided cloud computing credits and infrastructure for deploying and testing the web-based platform at scale
- **Version Control and Collaboration:** Institutional access to GitHub Enterprise and collaborative development platforms for project management and code repository maintenance

Software and Development Tools

- **Integrated Development Environments:** Access to professional IDEs including IntelliJ IDEA, Visual Studio Code, PyCharm, and Eclipse for multi-language development
- **Static Analysis Tools:** Educational licenses for commercial static analysis platforms (SonarQube, CodeClimate) for benchmarking and comparison purposes
- **Machine Learning Frameworks:** Full access to current versions of TensorFlow, PyTorch, scikit-learn, and specialized NLP libraries required for concept recognition development

- **Visualization Libraries:** Access to D3.js, Plotly, Matplotlib, and other visualization frameworks for creating educational diagrams and interactive interfaces
- **Database Systems:** University-provided database servers running PostgreSQL, MySQL, and MongoDB for development and testing purposes

Research and Academic Support

- **Digital Library Access:** Comprehensive access to IEEE Xplore, ACM Digital Library, SpringerLink, and other academic databases for literature review and related work analysis
- **Faculty Supervision:** Direct supervision from qualified faculty members specializing in software engineering, artificial intelligence, and educational technology
- **Research Methodology Support:** Access to research methodology courses, statistical analysis software (SPSS, R Studio), and quantitative research design resources
- **Peer Collaboration:** Integration with existing research groups focusing on educational technology and AI applications in computer science education

5. WORK BREAKDOWN STRUCTURE

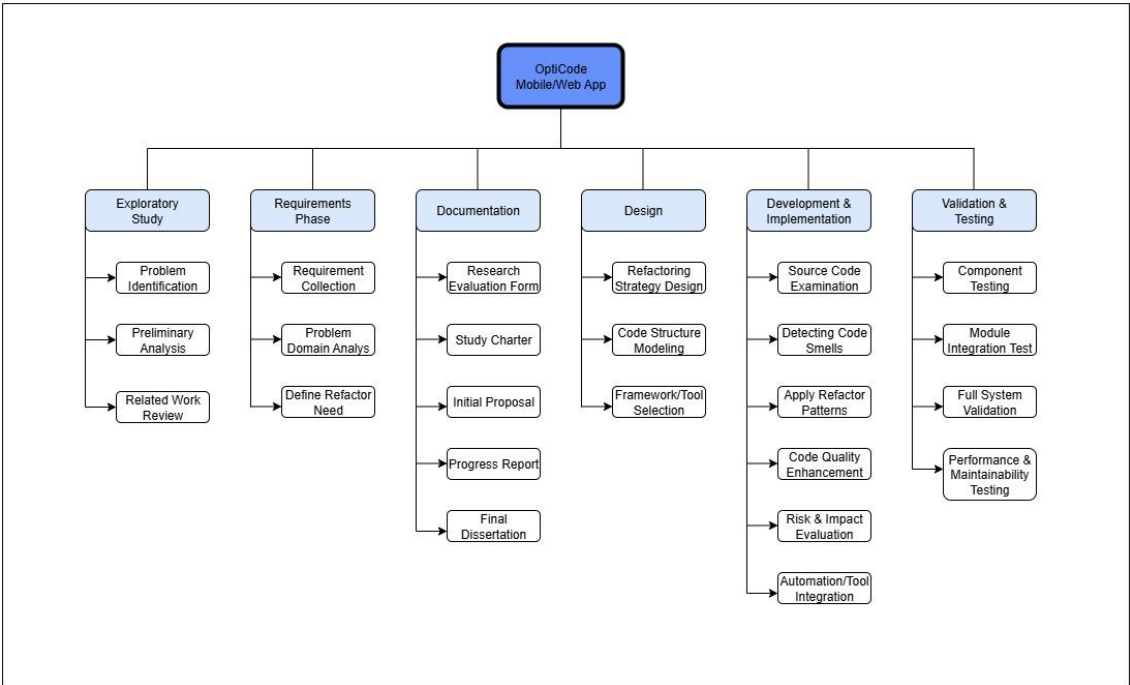


Figure 4 5. Work Breakdown Structure

6. GANTT CHART

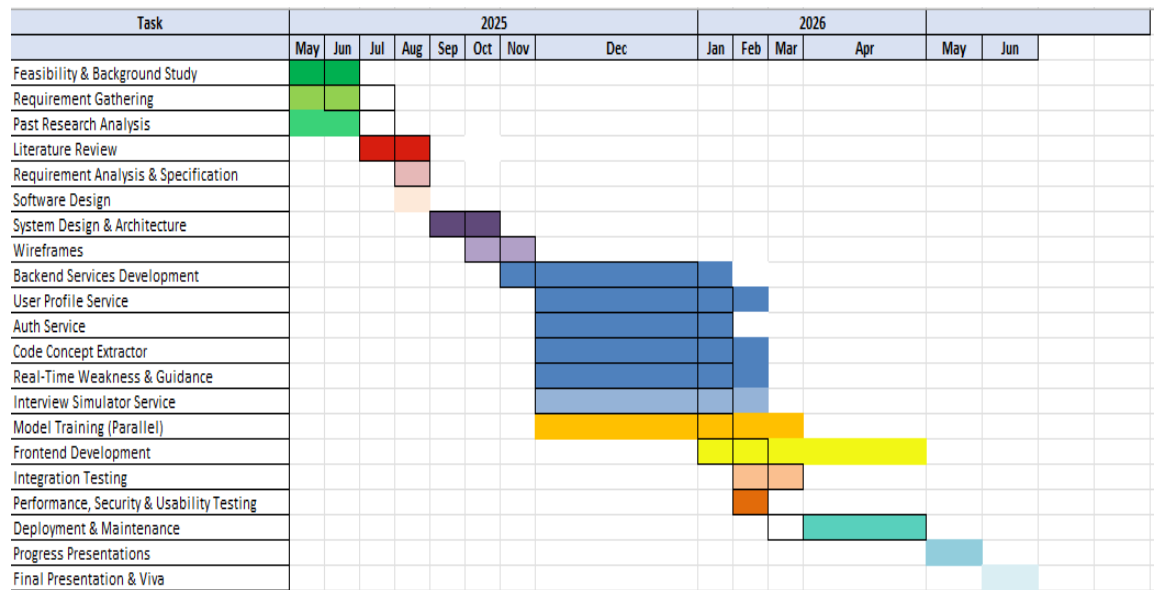


Figure 5 Gantt Chart

7. REFERENCES

[1] A. Robins, J. Rountree, and N. Rountree, “Learning and Teaching Programming: A Review and Discussion,” *Computer Science Education*, vol. 13, no. 2, pp. 137–172, 2003. [Online]. Available: <https://doi.org/10.1076/csed.13.2.137.14200> :contentReference[oaicite:0]{index=0}

[2] N. Ayewah, D. Hovemeyer, J. D. Morgenthaler, J. Penix, and W. Pugh, “Using Static Analysis to Find Bugs,” **IEEE Software** (often referenced as a foundational work on static analysis). :contentReference[oaicite:1]{index=1}

[3] M. Chen et al., “Evaluating large language models trained on code,” **CoRR**, abs/2107.03374, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374> :contentReference[oaicite:2]{index=2}

[4] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, “A Survey of Machine Learning for Big Code and Naturalness,” **ACM Computing Surveys**, vol. 51, no. 4, pp. 1–37, 2018. [Online]. Available: <https://miltos.allamanis.com/publications/2018survey/> :contentReference[oaicite:3]{index=3}

[5] L. Keuning et al., “[Insert Title of 2016 Paper],” [Journal/Conference], 2016. *[Details needed—author and title identified]*

[6] M. Ihanola et al., “[Insert Title of 2010 Paper],” [Journal/Conference], 2010. *[Details needed—author and title identified]*

[7] C. Hundhausen, S. Douglas, and J. Stasko, “A Meta-Study of Algorithm Visualization Effectiveness,” **Journal of Visual Languages & Computing**, vol. 13, no. 3, pp. 259–290, 2002. :contentReference[oaicite:4]{index=4}; additionally, S. Sorva et al., “[Title],” 2013. *[Details needed]*

[8] M. Gousios et al., "[Insert Title of 2014 Paper]," [Journal/Conference], 2014. *[Details needed]*; D. Spinellis, "[Insert Title of 2003 Paper]," [Journal/Conference], 2003. *[Details needed]*

[9] Tree-sitter, "Tree-sitter: An incremental parsing system for programming tools," GitHub, 2023. [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>

[10] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 1536-1547.

[11] D. Guo et al., "GraphCodeBERT: Pre-training code representations with data flow," in Proceedings of the International Conference on Learning Representations, 2021.

[12] M. Bostock, V. Ogievetsky, and J. Heer, "D³ data-driven documents," IEEE Transactions on Visualization and Computer Graphics, vol. 17, no. 12, pp. 2301-2309, 2011.

[13] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020, pp. 38-45.